Trees

Chapter 11

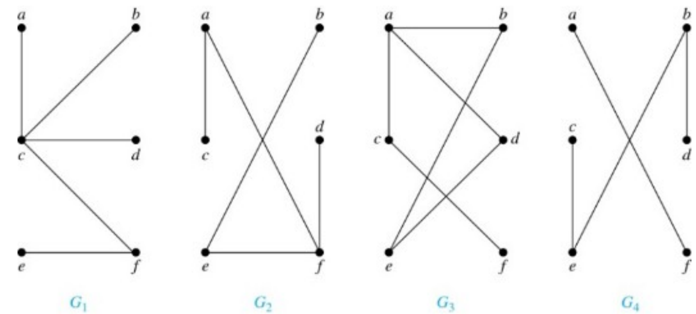
With Question/Answer Animations

Introduction to Trees

Section 11.1

Trees 1

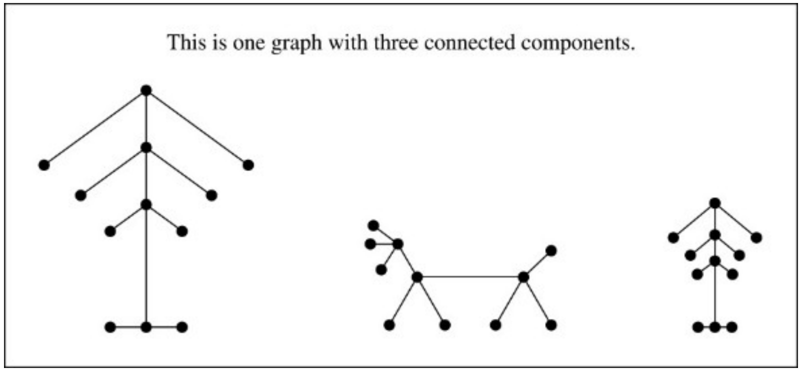
Definition: A *tree* is a connected undirected graph with no simple circuits回路.



Example: Which of these graphs are trees?

Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because *e, b, a, d, e* is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

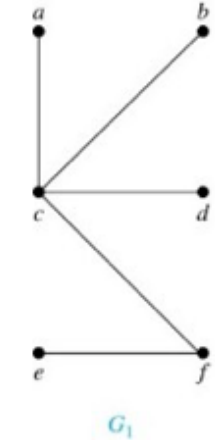
Definition: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree.



Jump to long description

Example of a forest

Trees 2



Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Proof: Assume that *T* is a tree. Then *T* is connected with no simple circuits. Hence, if *x* and *y* are distinct vertices of *T*, there is a simple path between them (by Theorem 1 of Section 10.4). This path must be unique - for if there were a second path, there would be a simple circuit in *T* (by Exercise 59 of Section 10.4). Hence, there is a unique simple path between any two vertices of a tree.

Now assume that there is a unique simple path between any two vertices of a graph *T*. Then *T* is connected because there is a path between any two of its vertices. Furthermore, *T* can have no simple circuits since if there were a simple circuit, there would be two paths between some two vertices. Then T is a tree.

Hence, a graph with a unique simple path between any two vertices is a tree.

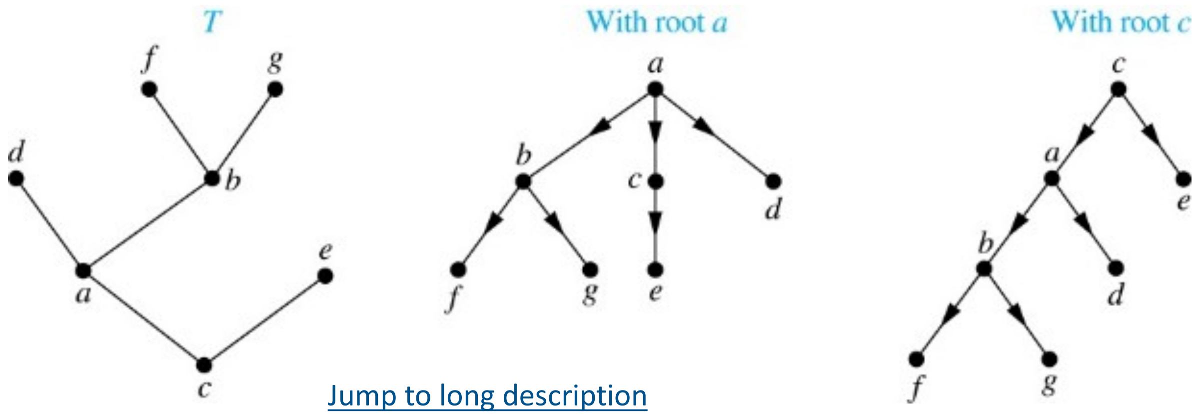
11.1.1 Rooted Trees

Definition: A *rooted tree* (根树)is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.

•***Applications of rooted trees***:

Decision trees, organization charts, tournament competition, database, computer programming (sorting and searching) etc.



Jump to long description

Rooted Tree Terminology

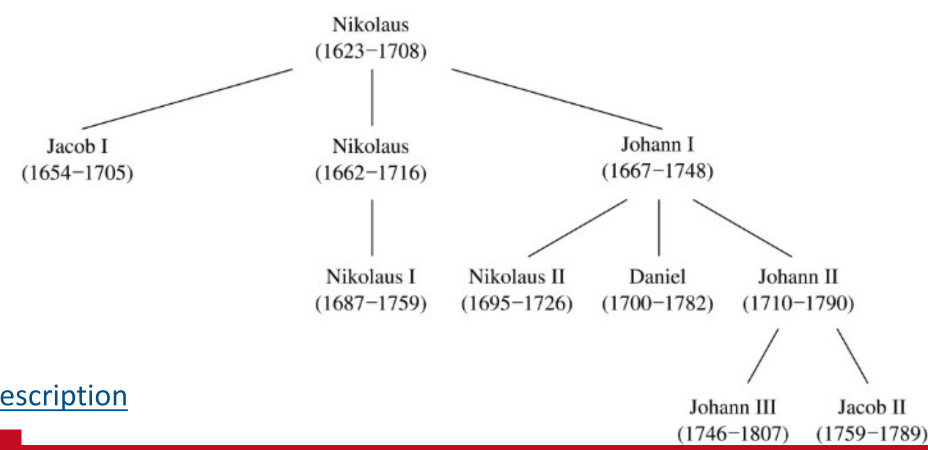
Terminology for rooted trees is a mix from botany 植物学and genealogy 家谱.

If *v* is a vertex of a rooted tree other than the root, the *parent* 父节点of *v* is the unique vertex *u* such that there is a directed edge from *u* to *v*. When *u* is a parent of *v*, *v* is called a *child* 结点of *u*. Vertices with the same parent are called *siblings*.兄弟姐妹

The *ancestors* 祖先of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The *descendants* 后代of a vertex *v* are those vertices that have *v* as an ancestor.

A vertex of a rooted tree with no children is called a *leaf* 叶. Vertices that have children are called *internal vertices*.内顶点 (root is also an internal vertices)

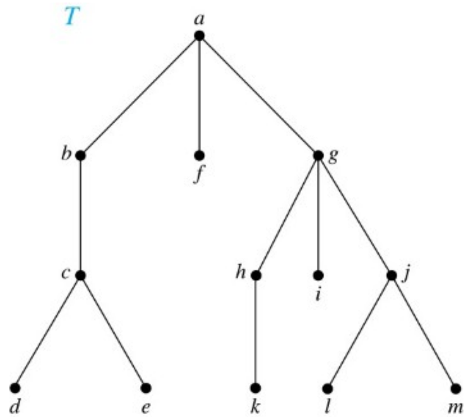
If *a* is a vertex in a tree, the *subtree* 子树with *a* as its root is the subgraph of the tree consisting of *a* and its descendants and all edges incident to these descendants.



Jump to long description

Terminology for Rooted Trees

Example: In the rooted tree *T* (with root *a*):



(i) Find the parent of *c*, the children of *g*, the siblings of *h*, the ancestors of *e*, and the descendants of *b*.

(ii) Find all internal vertices and all leaves.

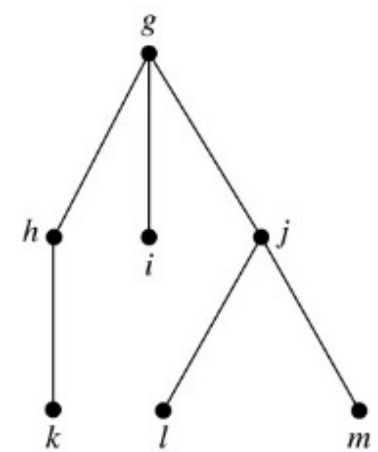
(iii) What is the subtree rooted at *g*?

Solution:

(i) The parent of c is b. The children of g are h, i, and j. The siblings of h are i and j. The ancestors of e are c, b, and a. The descendants of b are c, d, and e.

(ii) The internal vertices are a, b, c, g, h, and j. The leaves are d, e, f, i, k, l, and m.

(iii) We display the subtree rooted at g.

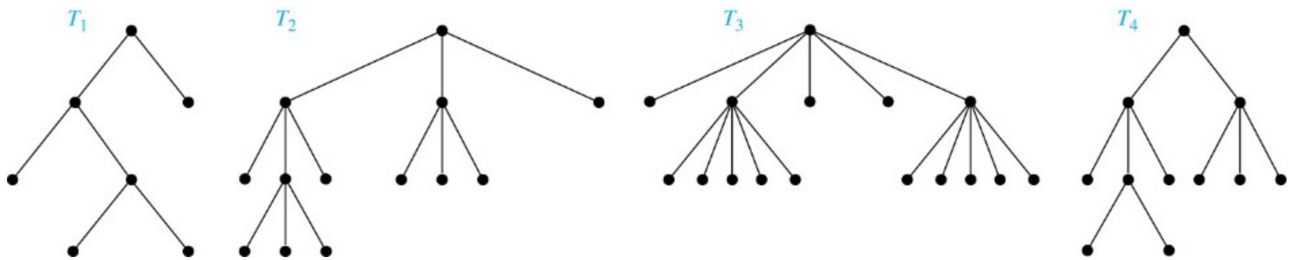


Jump to long description

m-ary Rooted Trees

Definition: A rooted tree is called an *m-ary tree (m元树)* if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree (完全m元树)* if every internal vertex has exactly *m* children. An *m*-ary tree with *m* = 2 is called a *binary* tree.(二元树for binary tree, full is not nessesary)

Example: Are the following rooted trees full *m*-ary trees for some positive integer *m*?



Solution: T1 is a full binary tree because each of its internal vertices has two children. T2 is a full 3-ary tree because each of its internal vertices has three children. In T3 each internal vertex has five children, so T3 is a full 5-ary tree. T4 is not a full m-ary tree for any m because some of its internal vertices have two children and others have three children.

Jump to long description

Ordered Rooted Trees

Definition: An *ordered rooted tree(有序有根树)* is a rooted tree where the children of each internal vertex are ordered.

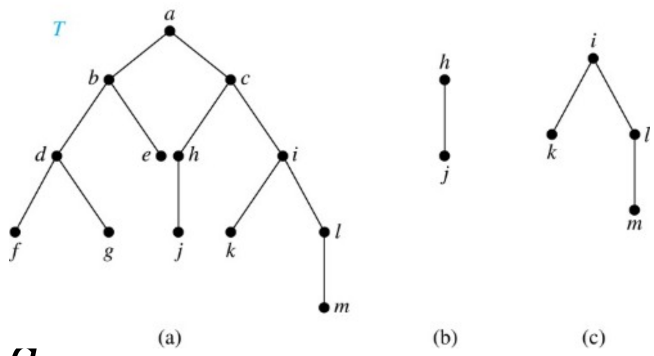
- We draw ordered rooted trees so that the children of each internal vertex are shown in order from left to right.

Definition: In an ordered binary tree (usually called just a binary tree), if an internal vertex of a binary tree has two children, the first is called the *left child (左子节点)* and the second the *right child (右子节点)*. The tree rooted at the left child of a vertex is called the *left subtree (左子树)* of this vertex, and the tree rooted at the right child of a vertex is called the *right subtree(右子树)* of this vertex.

Example: Consider the binary tree T.

I. What are the left and right children of d?

II. What are the left and right subtrees of c?



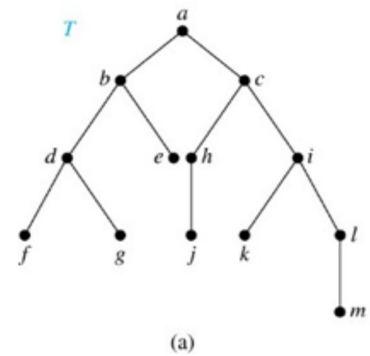
Solution:

I. The left child of d is f and the right child is g.

II. The left and right subtrees of c are displayed in (b) and (c).

Jump to long description

11.1.3 Properties of Trees



Theorem 2: A tree with n vertices has $n - 1$ edges.

***Proof* (*by mathematical induction*):**

BASIS STEP: When $n = 1$, a tree with one vertex has no edges. Hence, the theorem holds when $n = 1$.

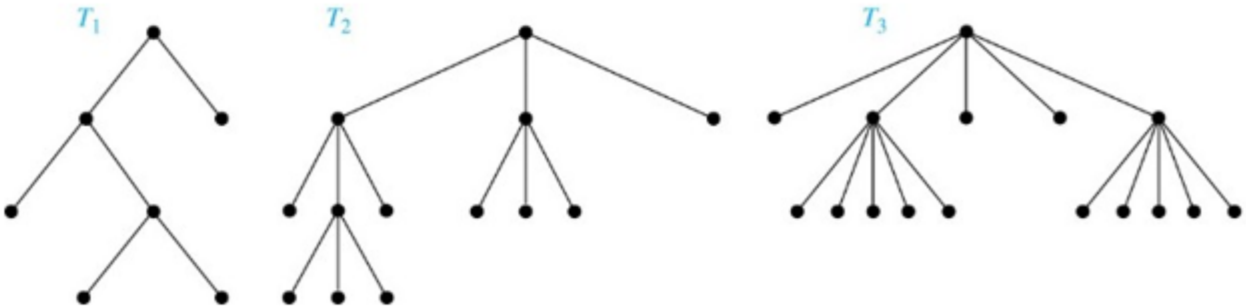
INDUCTIVE STEP: Assume that every tree with k vertices has $k - 1$ edges.

Suppose that a tree T has $k + 1$ vertices and that v is a leaf of T. Let w be the parent of v. Removing the vertex v and the edge connecting w to v produces a tree T' with k vertices. By the inductive hypothesis, T' has $k - 1$ edges. Because T has one more edge than T', we see that T has k edges. This completes the inductive step.

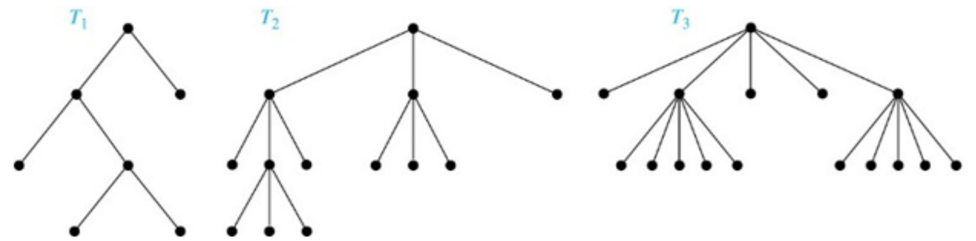
Counting Vertices in Full *m*-Ary Trees [1]

Theorem 3: A full *m*-ary tree with i internal vertices has $n = mi + 1$ vertices.

Proof : Every vertex, except the root, is the child of an internal vertex. Because each of the i internal vertices has m children, there are mi vertices in the tree other than the root. Hence, the tree contains $n = mi + 1$ vertices.



Counting Vertices in Full *m*-Ary Trees 2



Theorem 4: A full *m*-ary tree with

I. n vertices has $i = (n - 1)/m$ internal vertices and $l = [(m - 1)n + 1]/m$ leaves,

II. i internal vertices has $n = mi + 1$ vertices and $l = (m - 1)i + 1$ leaves,

III. l leaves has $n = (ml - 1)/(m - 1)$ vertices and $i = (l - 1)/(m - 1)$ internal vertices.

Proof (of part i): Solving for i in $n = mi + 1$ (from Theorem 3) gives $i = (n - 1)/m$. Since each vertex is either a leaf or an internal vertex, $n = l + i$. By solving for l and using the formula for i, we see that

$$l = n - i = n - (n-1)/m = \left[(m-1)n + 1\right]/m.$$

Level of vertices and height of trees

When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length.

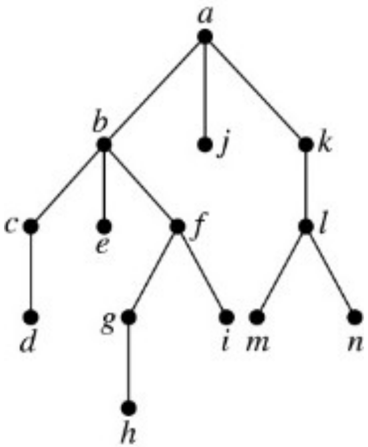
To make this idea precise we need some definitions:

- The *level* 层of a vertex *v* in a rooted tree is the length of the unique path from the root to this vertex.
- The *height* 高度of a rooted tree is the maximum of the levels of the vertices.

Example:

I. Find the level of each vertex in the tree to the right.

II. What is the height of the tree?



Solution:

I. The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1. Vertices *c*, *e*, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3. Vertex *h* is at level 4.

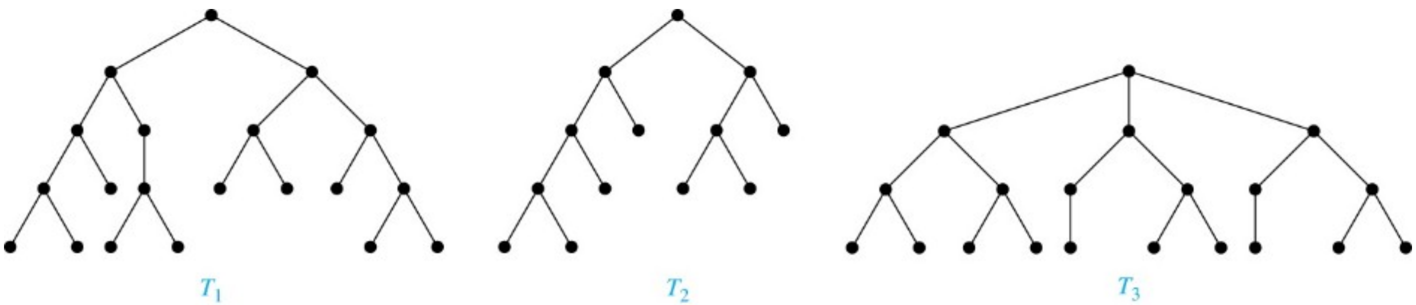
II. The height is 4, since 4 is the largest level of any vertex.

Jump to long description

Balanced *m*-Ary Trees

Definition: A rooted *m*-ary tree of height *h* is *balanced* 均衡if all leaves are at levels *h* or *h* – 1.

Example: Which of the rooted trees shown below is balanced?

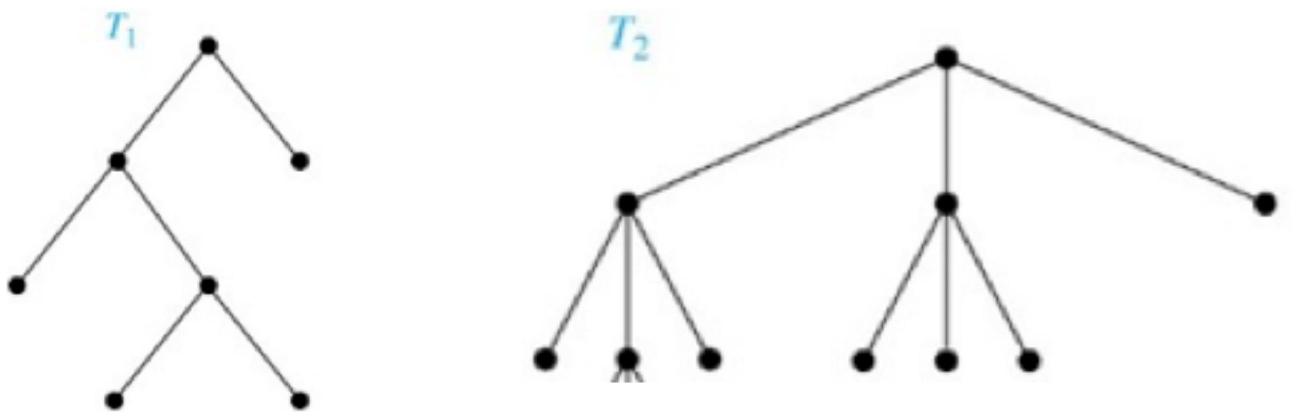


Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

Jump to long description

The Bound for the Number of Leaves in an m-Ary Tree

Theorem 5: There are at most m^h leaves in an m-ary tree of height h. (*see text for the proof*)



Corollary 1: If an m-ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$. If the m-ary tree is full and balanced, then $h = \lceil \log_m l \rceil$. (We are using the ceiling function here. Recall that $\lceil x \rceil$ is the smallest integer greater than or equal to x)

Proof: We know that $l \leq m^h$ from Theorem 5. Taking logarithms to the base m shows that $\log_m l \leq h$. Because h is an integer, we have $h \geq \lceil \log_m l \rceil$. Now suppose that the tree is balanced. Then each leaf is at level h or $h - 1$, and because the height is h, there is at least one leaf at level h. It follows that there must be more than m^{h-1} leaves (see Exercise 30). Because $l \leq m^h$, we have $m^{h-1} < l \leq m^h$. Taking logarithms to the base m in this inequality gives $h - 1 < \log_m l \leq h$. Hence, $h = \lceil \log_m l \rceil$. ◁

Section Summary[1]

1.Introduction to Trees: *tree, forest.*

2.Rooted Trees : *parent, child* 结点, *siblings*兄弟姐妹, *ancestors* 祖先, *descendants 后代, leaf 叶, internal vertices*内顶点, *subtree子树.*

A rooted tree is called an *m-ary tree (m元树)*if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree (完全m元树)*if every internal vertex has exactly *m* children. An *m*-ary tree with *m* = 2 is called a *binary* tree.(二元树)

ordered rooted tree(有序有根树) binary tree : left child (左子节点) right child (右子节点). left subtree (左子树) right subtree(右子树) .

3.Properties of Trees

A tree with n vertices has $n - 1$ edges.

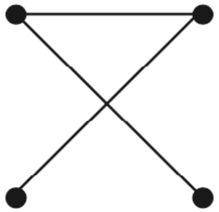
A full *m*-ary tree with i internal vertices has $n = mi + 1$ vertices.

Level of vertices and height of trees. Balanced *m*-Ary Trees.

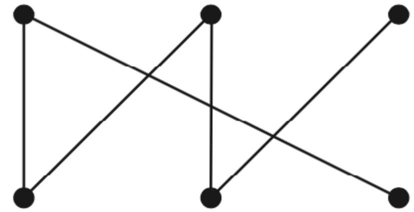
There are at most m^h leaves in an *m*-ary tree of height h.

2. Which of these graphs are trees?

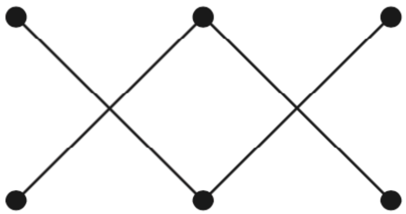
a)



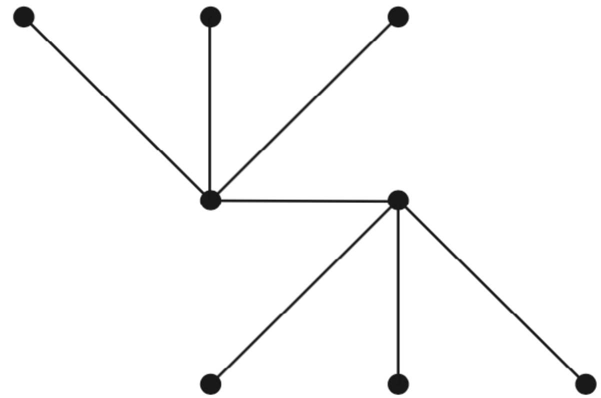
b)



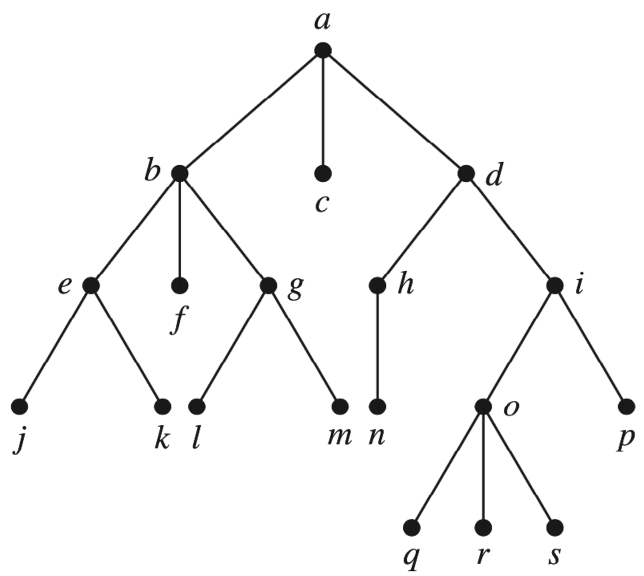
c)



d)



4. Answer the same questions as listed in Exercise 3 for the rooted tree illustrated.



a) Which vertex is the root?

b) Which vertices are internal?

c) Which vertices are leaves?

d) Which vertices are children of j?

e) Which vertex is the parent of h?

f) Which vertices are siblings of o?

g) Which vertices are ancestors of m?

h) Which vertices are descendants of b?

6. Is the rooted tree in Exercise 4 a full m-ary tree for some positive integer m?

8. What is the level of each vertex of the rooted tree in Exercise 4?

10. Draw the subtree of the tree in Exercise 4 that is rooted at

a) a. **b)** c. **c)** e.

19. How many edges does a full binary tree with 1000 internal vertices have?

20. How many leaves does a full 3-ary tree with 100 vertices have?

Exercise

Page 791 1bcd 3 5 7 9b 17 18